

Question 1: Finding Solutions of Linear Systems

Part 1: REF and RREF Algorithm Implementation

Mathematical Concepts

- **Augmented Matrix** $[A \mid b]$: Combines coefficient matrix $A \in \mathbb{R}^{m \times n}$ and constant vector $b \in \mathbb{R}^{m \times 1}$ into an $m \times (n+1)$ system.
- **Row Echelon Form (REF)**: A matrix form where leading coefficients descend in a staircase pattern, and all elements directly below each pivot are zeroed via forward elimination.
- **Reduced Row Echelon Form (RREF)**: A canonical form where each pivot equals 1 and is the unique non-zero entry in its column (transforming pivot columns into standard unit basis vectors e_i).
- **Handling Division by Zero**: Explicit checks ensure zero pivots are avoided during forward row operations and back-substitution scaling.

```
import random

# Set random seed for reproducible float generation
random.seed(42)

def generate_8digit_float():
    """Generates random floats formatted as n.dddddddd per Assignment
    Instruction 6."""
    return round(random.uniform(1.0, 9.0) + random.random(), 8)

def construct_augmented_matrix(A, b):
    """Constructs augmented matrix  $[A \mid b]$  without using external
    linear algebra libraries."""
    m = len(A)
    return [A[i][:] + [b[i]] for i in range(m)]

def compute_ref(augmented_mat, tol=1e-10):
    """Computes Row Echelon Form (REF) via forward Gaussian
    elimination."""
    M = [row[:] for row in augmented_mat]
    m = len(M)
    n = len(M[0]) - 1

    pivot_row = 0
```

```

for col in range(n):
    if pivot_row >= m:
        break

    # Partial pivoting: select largest magnitude entry in the
current column
    max_row = pivot_row
    max_val = abs(M[pivot_row][col])
    for r in range(pivot_row + 1, m):
        if abs(M[r][col]) > max_val:
            max_val = abs(M[r][col])
            max_row = r

    # EXPLICIT CHECK: Detect and avoid division by zero
# In case all elements in the column below the current row are
~0, division by 0 would occur.
# We skip this column or alternative inputs can be selected.
    if max_val < tol:
        continue

    # Swap rows to position the pivot
    M[pivot_row], M[max_row] = M[max_row], M[pivot_row]

    # Forward elimination below pivot
    for r in range(pivot_row + 1, m):
        # Explicit check to safeguard against division by zero
        if abs(M[pivot_row][col]) < tol:
            continue

        factor = M[r][col] / M[pivot_row][col]
        M[r][col] = 0.0
        for c in range(col + 1, n + 1):
            M[r][c] -= factor * M[pivot_row][c]

    pivot_row += 1

return M

def compute_rref(ref_mat, tol=1e-10):
    """Computes Reduced Row Echelon Form (RREF) by back-substitution
and scaling."""
    M = [row[:] for row in ref_mat]
    m = len(M)
    n = len(M[0]) - 1

    for r in range(m - 1, -1, -1):
        pivot_col = -1
        for c in range(n):
            if abs(M[r][c]) > tol:

```

```

        pivot_col = c
        break

    if pivot_col == -1:
        continue

    pivot_val = M[r][pivot_col]

    # EXPLICIT CHECK: Avoid division by zero prior to row
normalisation
    if abs(pivot_val) < tol:
        continue

    # Scale row so that pivot value equals 1.0
    for c in range(pivot_col, n + 1):
        M[r][c] /= pivot_val

    # Eliminate entries above the pivot
    for above_r in range(r):
        factor = M[above_r][pivot_col]
        for c in range(pivot_col, n + 1):
            M[above_r][c] -= factor * M[r][c]

    return M

# -----
# DEMONSTRATION & DETAILED IN-OUTPUT ANALYSIS
# -----
# Create a 3x4 underdetermined system ( $m < n$ ) with 8-decimal float
entries
m_demo, n_demo = 3, 4
A_demo = [
    [generate_8digit_float() for _ in range(n_demo)] for _ in
range(m_demo)
]
b_demo = [generate_8digit_float() for _ in range(m_demo)]

aug_demo = construct_augmented_matrix(A_demo, b_demo)
ref_demo = compute_ref(aug_demo)
rref_demo = compute_rref(ref_demo)

def print_formatted_matrix(mat, label):
    print(f"\n{label}:")
    for row in mat:
        print(" [ " + ", ".join(f"{v:12.8f}" for v in row) + " ]")

print("=" * 80)

```

```

print("                QUESTION 1 (PART 1): REF & RREF EXPERIMENT")
print("=" * 80)

print_formatted_matrix(A_demo, "1. Coefficient Matrix A (3 x 4)")
print("\n2. Target Vector b (3 x 1):")
print("  [ " + ", ".join(f"{v:12.8f}" for v in b_demo) + " ]")

print_formatted_matrix(aug_demo, "3. Augmented Matrix [A | b] (3 x 5)")
print_formatted_matrix(ref_demo, "4. Row Echelon Form (REF)")
print_formatted_matrix(rref_demo, "5. Reduced Row Echelon Form (RREF)")

print("\n" + "=" * 80)
print("                MATHEMATICAL OBSERVATIONS")
print("=" * 80)
print(
    "* Augmented Matrix Formation: Vector b is successfully concatenated as the 5th column."
)
print(
    "* REF Conversion: Forward elimination produced zero entries below all leading pivots,"
)
print("  yielding an upper-staircase structure without runtime zero division.")
print(
    "* RREF Conversion: All leading pivot entries are scaled to exactly 1.00000000,"
)
print(
    "  and all entries above and below every pivot are eliminated to 0.00000000."
)
print(
    "* Canonical Structure: The pivot columns match the standard unit basis vectors e_i."
)
print("=" * 80)

```

```

=====
=====
                QUESTION 1 (PART 1): REF & RREF EXPERIMENT
=====
=====

```

```

1. Coefficient Matrix A (3 x 4):
  [ 6.14042514, 3.42344529, 7.56846920, 8.22437537 ]
  [ 4.40517178, 3.25445909, 1.41112541, 6.74401698 ]
  [ 3.35279066, 7.48194241, 8.14469341, 3.87748363 ]

```

```

2. Target Vector b (3 x 1):
  [ 8.99429912, 1.83868312, 8.38368096 ]

3. Augmented Matrix [A | b] (3 x 5):
  [ 6.14042514, 3.42344529, 7.56846920, 8.22437537,
8.99429912 ]
  [ 4.40517178, 3.25445909, 1.41112541, 6.74401698,
1.83868312 ]
  [ 3.35279066, 7.48194241, 8.14469341, 3.87748363,
8.38368096 ]

4. Row Echelon Form (REF):
  [ 6.14042514, 3.42344529, 7.56846920, 8.22437537,
8.99429912 ]
  [ 0.00000000, 5.61267520, 4.01216313, -0.61318408,
3.47262002 ]
  [ 0.00000000, 0.00000000, -4.58930488, 0.93104076, -
5.10788900 ]

5. Reduced Row Echelon Form (RREF):
  [ 1.00000000, 0.00000000, 0.00000000, 1.56949145,
0.19155515 ]
  [ 0.00000000, 1.00000000, 0.00000000, 0.03577098, -
0.17690515 ]
  [ 0.00000000, 0.00000000, 1.00000000, -0.20287185,
1.11299840 ]

```

```

=====
=====
                                MATHEMATICAL OBSERVATIONS
=====
=====
* Augmented Matrix Formation: Vector b is successfully concatenated as
the 5th column.
* REF Conversion: Forward elimination produced zero entries below all
leading pivots,
  yielding an upper-staircase structure without runtime zero division.
* RREF Conversion: All leading pivot entries are scaled to exactly
1.00000000,
  and all entries above and below every pivot are eliminated to
0.00000000.
* Canonical Structure: The pivot columns match the standard unit basis
vectors e_i.
=====
=====

```

Part 2: Pivot & Non-Pivot Identification, Particular Solution, and Homogeneous Null Space ($Ax=0$)

Theoretical Foundations (From Course Notes)

- **Pivot Variables (Basic Variables):** The leading variables corresponding to columns in RREF containing standard unit basis vectors e_i .
- **Non-Pivot Variables (Free Variables):** Variables that can be assigned arbitrary values because their corresponding columns lack leading pivots.
- **Particular Solution (x_p):** Formed by setting all free variables to 0, enabling the basic variables to match the augmented constant column directly.
- **Homogeneous Solutions ($Ax=0$):** The basis vectors spanning the null space (kernel) of A . For each free variable x_f , we set $x_f=1$ (and all other free variables to 0), solving for the pivot variables via:

$$x_{\text{pivot}} = - \sum_{f \in \text{Free}} \text{RREF}[\text{pivot_row}, f] \cdot x_f$$

```
#
=====
===
# QUESTION 1 (PART 2): EXTRACTION OF PIVOTS, PARTICULAR & NULL SPACE BASIS
#
=====
===

def identify_and_extract_solutions(rref_mat, tol=1e-10):
    """Parses an RREF matrix to identify pivot/non-pivot columns,
    constructs the particular solution xp, and determines basis
    vectors for Ax = 0.
    """
    m = len(rref_mat)
    n = len(rref_mat[0]) - 1 # Number of unknown variables (columns
of A)

    pivot_columns = []
    pivot_row_mapping = (
        {}
    ) # Maps pivot column index -> row index containing the pivot

    # 1. Identify Pivot Columns (Columns containing the leading 1 for
each row)
    for r in range(m):
```

```

        for c in range(n):
            if abs(rref_mat[r][c] - 1.0) < tol:
                # Confirm it is the leading entry in row r
                is_leading = True
                for prev_c in range(c):
                    if abs(rref_mat[r][prev_c]) > tol:
                        is_leading = False
                        break
                if is_leading and c not in pivot_columns:
                    pivot_columns.append(c)
                    pivot_row_mapping[c] = r
                    break

# 2. Identify Non-Pivot Columns (Free variables)
free_columns = [c for c in range(n) if c not in pivot_columns]

# 3. Construct Particular Solution (xp): Set all free variables to
0
x_particular = [0.0] * n
for p_col in pivot_columns:
    row_idx = pivot_row_mapping[p_col]
    x_particular[p_col] = rref_mat[row_idx][n]

# 4. Construct Basis Vectors for Homogeneous System Ax = 0
# For each free variable, set it to 1 and express pivot variables
in terms of it
null_space_basis = []
for free_c in free_columns:
    v_homogeneous = [0.0] * n
    v_homogeneous[free_c] = 1.0 # Set current free variable to
1.0

    for p_col in pivot_columns:
        row_idx = pivot_row_mapping[p_col]
        v_homogeneous[p_col] = -rref_mat[row_idx][free_c]
    null_space_basis.append(v_homogeneous)

return pivot_columns, free_columns, x_particular, null_space_basis

#
-----
---
# DEMONSTRATION & IN-OUTPUT OBSERVATIONS
#
-----
---
# We apply the extraction on the computed RREF from Part 1
pivot_cols, free_cols, x_part, null_basis =
identify_and_extract_solutions(
    rref_demo

```

```

)

print("=" * 80)
print("          QUESTION 1 (PART 2): SYSTEM SOLUTION EXTRACTION &
OBSERVATIONS")
print("=" * 80)

print(f"\n1. IDENTIFIED COLUMN CLASSIFICATION:")
print(f"    * Total Variables (n)          : {n_demo}")
print(f"    * Pivot Columns (0-indexed): {pivot_cols} --> Basic
Variables")
print(f"    * Free Columns (0-indexed): {free_cols} --> Free
Variables")
print(
    f"    * Rank of Matrix (rk(A))      : {len(pivot_cols)} (Number of
leading pivots)"
)
print(
    f"    * Nullity of Matrix              : {len(free_cols)} (Dimension of
Null Space ker(A))"
)

print("\n2. PARTICULAR SOLUTION VECTOR (x_p):")
print("    [ " + ", ".join(f"{v:12.8f}" for v in x_part) + " ]")

print("\n3. HOMOGENEOUS BASIS VECTORS (Solutions to Ax = 0):")
for idx, v_null in enumerate(null_basis):
    print(
        f"    * Vector v_{idx+1} (Free col index {free_cols[idx]}): [ "
        + ", ".join(f"{v:12.8f}" for v in v_null)
        + " ]"
    )

print("\n" + "=" * 80)
print("          MATHEMATICAL OBSERVATIONS")
print("=" * 80)
print(
    f"* Basic vs Free Partitioning: Columns {pivot_cols} contain the
pivots 1.0, defining the"
)
print(
    f"    basic variables. Column {free_cols} has no pivot, providing
{len(free_cols)} degree(s) of freedom."
)
print(
    f"* Particular Solution Formulation: Evaluated by setting free
variable x_3 = 0,"
)
print(
    f"    allowing basic variables [x_0, x_1, x_2] to be directly"
)

```

```

assigned the augmented values."
)
print(
    "* Null Space Basis ( $Ax = 0$ ): Setting free variable  $x_3 = 1$ 
isolates the exact linear"
)
print(
    "    dependency ratio required for the pivot columns to cancel the
non-pivot column to 0."
)
print(
    "* Dimension Consistency: Rank-Nullity Theorem verified:  $\text{rank}(A) +$ 
 $\text{nullity}(A) = 3 + 1 = 4 = n$ ."
)
print("=" * 80)

```

```

=====
QUESTION 1 (PART 2): SYSTEM SOLUTION EXTRACTION & OBSERVATIONS
=====

```

```

1. IDENTIFIED COLUMN CLASSIFICATION:
* Total Variables (n)      : 4
* Pivot Columns (0-indexed): [0, 1, 2] --> Basic Variables
* Free Columns  (0-indexed): [3]  --> Free Variables
* Rank of Matrix (rk(A))   : 3 (Number of leading pivots)
* Nullity of Matrix       : 1 (Dimension of Null Space  $\ker(A)$ )

```

```

2. PARTICULAR SOLUTION VECTOR ( $x_p$ ):
[ 0.19155515, -0.17690515, 1.11299840, 0.00000000 ]

```

```

3. HOMOGENEOUS BASIS VECTORS (Solutions to  $Ax = 0$ ):
* Vector  $v_1$  (Free col index 3): [ -1.56949145, -0.03577098,
0.20287185, 1.00000000 ]

```

```

=====
MATHEMATICAL OBSERVATIONS
=====

```

```

* Basic vs Free Partitioning: Columns [0, 1, 2] contain the pivots
1.0, defining the
    basic variables. Column [3] has no pivot, providing 1 degree(s) of
freedom.
* Particular Solution Formulation: Evaluated by setting free variable
 $x_3 = 0$ ,
    allowing basic variables [ $x_0, x_1, x_2$ ] to be directly assigned the
augmented values.
* Null Space Basis ( $Ax = 0$ ): Setting free variable  $x_3 = 1$  isolates

```

the exact linear dependency ratio required for the pivot columns to cancel the non-pivot column to 0.

* Dimension Consistency: Rank-Nullity Theorem verified: $\text{rank}(A) + \text{nullity}(A) = 3 + 1 = 4 = n$.

=====

Part 3: Complete Demonstration on a 5×7 Random Float System & General Solution Verification (2 Marks)

Mathematical Concepts & Requirements (From Course Slides & MML)

- **Underdetermined System** ($m < n$): A system with $m=5$ equations and $n=7$ unknown variables.
- **Assignment Instruction 6:** All numerical entities are random floats formatted as `n.dddddddd` (8 decimal places).
- **Minus-1 Concept / Linear Combination Principle:** Non-pivot columns are expressed as linear combinations of pivot columns to find null space vectors $Ax=0$.
- **General Solution** (x_{gen}): The complete affine solution space given by:

$$x_{\text{gen}} = x_p + \sum_{k=1}^{n - \text{rk}(A)} \lambda_k v_k, \lambda_k \in R$$

- **Numerical Verification:** Direct computation of residual $\|Ax_{\text{gen}} - b\|_{\infty}$ to ensure compatibility within floating-point epsilon.

```
#
=====
==
# QUESTION 1 (PART 3): COMPLETE 5x7 DEMONSTRATION & VERIFICATION
#
=====
==
import random

# Fixed seed for strict numerical reproducibility
random.seed(2026)

def generate_8digit_float():
    """Generates random floats formatted strictly as n.dddddddd
    (Assignment Instruction 6)."""
    return round(random.uniform(1.0, 9.0) + random.random(), 8)
```

```

# 1. Instantiate 5x7 Matrix A and 5x1 Vector b
m, n = 5, 7
A_5x7 = [[generate_8digit_float() for _ in range(n)] for _ in
range(m)]
b_5x1 = [generate_8digit_float() for _ in range(m)]

# 2. Build Augmented Matrix [A | b]
aug_5x7 = construct_augmented_matrix(A_5x7, b_5x1)

# 3. Compute Forward Elimination (REF) and Back-Substitution (RREF)
ref_5x7 = compute_ref(aug_5x7)
rref_5x7 = compute_rref(ref_5x7)

# 4. Extract Solution Subspaces
pivot_cols, free_cols, x_p, null_basis =
identify_and_extract_solutions(
    rref_5x7
)

# 5. Formulate General Solution with Arbitrary 8-Decimal Scalars
(lambda_1, lambda_2)
lambda_scalars = [3.14159265, -2.71828182]
x_gen = x_p[:]
for idx, vec in enumerate(null_basis):
    for j in range(n):
        x_gen[j] += lambda_scalars[idx] * vec[j]

# 6. Manual Matrix-Vector Multiplication for Verification (No built-in
solvers)
def manual_mat_vec(mat, vec):
    return [sum(mat[i][j] * vec[j] for j in range(len(vec))) for i in
range(len(mat))]

Ax_calc = manual_mat_vec(A_5x7, x_gen)
residuals = [abs(Ax_calc[i] - b_5x1[i]) for i in range(m)]
max_residual_err = max(residuals)

#
-----
---
# STRUCTURED PRINTER & DETAILED IN-OUTPUT OBSERVATIONS
#
-----
---
def print_mat(mat, title):
    print(f"\n{title}:")
    for row in mat:

```

```

        print(" [ " + ", ".join(f"{v:12.8f}" for v in row) + " ]")

print("=" * 85)
print("      QUESTION 1 (PART 3): FULL 5 x 7 SYSTEM DEMONSTRATION &
VERIFICATION")
print("=" * 85)

print_mat(A_5x7, "1. COEFFICIENT MATRIX A (5 x 7)")
print(
    f"\n2. TARGET VECTOR b (5 x 1):\n [ "
    + ", ".join(f"{v:12.8f}" for v in b_5x1)
    + " ]"
)
print_mat(aug_5x7, "3. AUGMENTED MATRIX [A | b] (5 x 8)")
print_mat(ref_5x7, "4. ROW ECHELON FORM (REF)")
print_mat(rref_5x7, "5. REDUCED ROW ECHELON FORM (RREF)")

print("\n" + "-" * 85)
print("6. COLUMN CLASSIFICATION & SUBSPACE DIMENSIONS:")
print(
    f"      * Pivot Columns (0-indexed)      : {pivot_cols} --> Basic
Variables (Rank = {len(pivot_cols)})"
)
print(
    f"      * Non-Pivot Columns (0-indexed): {free_cols} --> Free
Variables (Nullity = {len(free_cols)})"
)
print("-" * 85)

print(
    f"\n7. PARTICULAR SOLUTION (x_p):\n [ "
    + ", ".join(f"{v:12.8f}" for v in x_p)
    + " ]"
)

print("\n8. HOMOGENEOUS SOLUTIONS (Null Space Basis Vectors for Ax =
0):")
for idx, vec in enumerate(null_basis):
    print(
        f"      * v_{idx+1} (Free col {free_cols[idx]}): [ "
        + ", ".join(f"{v:12.8f}" for v in vec)
        + " ]"
    )

print(f"\n9. CHOSEN FREE SCALARS (lambda): {lambda_scalars}")
print(
    f"10. CONSTRUCTED GENERAL SOLUTION (x_gen = x_p + sum(lambda_k *
v_k)):\n [ "
    + ", ".join(f"{v:12.8f}" for v in x_gen)

```

```

    + " ]"
)

print("\n" + "=" * 85)
print("11. VERIFICATION OF GENERAL SOLUTION: (A * x_gen vs Target b)")
print("=" * 85)
print(
    f"    * Computed A * x_gen : [ "
    + ", ".join(f"{v:12.8f}" for v in Ax_calc)
    + " ]"
)
print(
    f"    * Target b           : [ "
    + ", ".join(f"{v:12.8f}" for v in b_5x1)
    + " ]"
)
print(f"    * Maximum Absolute Residual ||A*x_gen - b||_inf:
{max_residual_err:.14e}")

if max_residual_err < 1e-8:
    print(
        "    * Status: >> VERIFICATION SUCCESSFUL (Residual strictly
below tolerance 1e-8) <<"
    )
else:
    print("    * Status: >> VERIFICATION FAILED <<")

print("\n" + "=" * 85)
print("                                MATHEMATICAL OBSERVATIONS")
print("=" * 85)
print(
    f"* Rank & Degrees of Freedom: For m = {m}, n = {n}, rank(A) =
{len(pivot_cols)}. By Rank-Nullity Theorem,"
)
print(
    f"    dim(ker(A)) = {n} - {len(pivot_cols)} = {len(free_cols)} free
variable(s), generating an affine hyperplane of dimension
{len(free_cols)}."
)
print(
    f"* Particular Solution Verification: Setting free variables
[x_{free_cols[0]}, x_{free_cols[1]}] = 0 produces x_p"
)
print(
    "    directly matching the transformed RHS vector in the RREF
augmented column."
)
print(
    f"* Null Space Basis (Ax = 0): The vectors v_1 and v_2 form a
linearly independent basis for ker(A)."
```

```

)
print(
    f" For any scalars  $\lambda_1 = \{\lambda_{scalars}[0]\}$  and  $\lambda_2 = \{\lambda_{scalars}[1]\}$ ,  $A(\lambda_1 v_1 + \lambda_2 v_2) = 0.$ "
)
print(
    f"* Precision & Exactness: The maximum discrepancy between  $Ax_{gen}$  and  $b$  is  $\sim \{\max\_residual\_err:.2e\},$ "
)
print(
    " which proves numerical stability and correctness of the Gaussian elimination algorithm."
)
print("=" * 85)

```

===== QUESTION 1 (PART 3): FULL 5 x 7 SYSTEM DEMONSTRATION & VERIFICATION =====

1. COEFFICIENT MATRIX A (5 x 7):

```

[ 2.45547483, 5.95458229, 2.04437936, 6.36480424,
7.81479851, 7.61242726, 7.59472851 ]
[ 3.53367692, 2.70265370, 5.41200135, 7.16081348,
2.93331039, 9.76861608, 8.44074333 ]
[ 5.18502027, 4.28060711, 3.79442279, 8.71577402,
5.64412617, 7.05044090, 4.77651330 ]
[ 6.18419761, 2.43794230, 7.60133957, 5.84500032,
3.48911361, 8.88905493, 5.16927402 ]
[ 2.92519780, 2.09576430, 8.19717364, 6.44851789,
5.08583219, 8.71404020, 5.12377680 ]

```

2. TARGET VECTOR b (5 x 1):

```

[ 9.42294314, 5.92485419, 6.66565873, 7.90031563,
5.71559334 ]

```

3. AUGMENTED MATRIX [A | b] (5 x 8):

```

[ 2.45547483, 5.95458229, 2.04437936, 6.36480424,
7.81479851, 7.61242726, 7.59472851, 9.42294314 ]
[ 3.53367692, 2.70265370, 5.41200135, 7.16081348,
2.93331039, 9.76861608, 8.44074333, 5.92485419 ]
[ 5.18502027, 4.28060711, 3.79442279, 8.71577402,
5.64412617, 7.05044090, 4.77651330, 6.66565873 ]
[ 6.18419761, 2.43794230, 7.60133957, 5.84500032,
3.48911361, 8.88905493, 5.16927402, 7.90031563 ]
[ 2.92519780, 2.09576430, 8.19717364, 6.44851789,
5.08583219, 8.71404020, 5.12377680, 5.71559334 ]

```

4. ROW ECHELON FORM (REF):

```
[ 6.18419761, 2.43794230, 7.60133957, 5.84500032,
3.48911361, 8.88905493, 5.16927402, 7.90031563 ]
[ 0.00000000, 4.98658186, -0.97378066, 4.04400984,
6.42942385, 4.08297170, 5.54223552, 6.28607278 ]
[ 0.00000000, 0.00000000, 4.78572009, 2.91934506,
2.22011823, 3.73762916, 1.63102817, 0.79042618 ]
[ 0.00000000, 0.00000000, 0.00000000, 3.30799896,
0.82873874, -0.56078592, -1.31332166, -2.42382871 ]
[ 0.00000000, 0.00000000, 0.00000000, 0.00000000, -
1.85205398, 2.91355559, 4.35472480, 0.97054363 ]
```

5. REDUCED ROW ECHELON FORM (RREF):

```
[ 1.00000000, 0.00000000, 0.00000000, 0.00000000,
0.00000000, -0.73226916, -1.30775450, 0.17355904 ]
[ 0.00000000, 1.00000000, 0.00000000, 0.00000000,
0.00000000, 2.93325784, 4.24399835, 2.57538117 ]
[ 0.00000000, 0.00000000, 1.00000000, 0.00000000,
0.00000000, 1.37378471, 1.31443803, 0.77514753 ]
[ 0.00000000, 0.00000000, 0.00000000, 1.00000000,
0.00000000, 0.22458986, 0.19204582, -0.60143292 ]
[ 0.00000000, 0.00000000, 0.00000000, 0.00000000,
1.00000000, -1.57314831, -2.35129475, -0.52403636 ]
```

6. COLUMN CLASSIFICATION & SUBSPACE DIMENSIONS:

- * Pivot Columns (0-indexed) : [0, 1, 2, 3, 4] --> Basic Variables (Rank = 5)
- * Non-Pivot Columns (0-indexed): [5, 6] --> Free Variables (Nullity = 2)

7. PARTICULAR SOLUTION (x_p):

```
[ 0.17355904, 2.57538117, 0.77514753, -0.60143292, -
0.52403636, 0.00000000, 0.00000000 ]
```

8. HOMOGENEOUS SOLUTIONS (Null Space Basis Vectors for $Ax = 0$):

- * v_1 (Free col 5): [0.73226916, -2.93325784, -1.37378471, -0.22458986, 1.57314831, 1.00000000, 0.00000000]
- * v_2 (Free col 6): [1.30775450, -4.24399835, -1.31443803, -0.19204582, 2.35129475, 0.00000000, 1.00000000]

9. CHOSEN FREE SCALARS (λ): [3.14159265, -2.71828182]

10. CONSTRUCTED GENERAL SOLUTION ($x_{gen} = x_p + \sum(\lambda_k * v_k)$):

```
[ -1.08079484, 4.89666345, 0.03228856, -0.78496810, -
1.97332696, 3.14159265, -2.71828182 ]
```

=====

```

=====
11. VERIFICATION OF GENERAL SOLUTION: (A * x_gen vs Target b)
=====
=====
* Computed A * x_gen : [ 9.42294314, 5.92485419, 6.66565873,
7.90031563, 5.71559334 ]
* Target b : [ 9.42294314, 5.92485419, 6.66565873,
7.90031563, 5.71559334 ]
* Maximum Absolute Residual ||A*x_gen - b||_inf: 1.24344978758018e-
14
* Status: >> VERIFICATION SUCCESSFUL (Residual strictly below
tolerance 1e-8) <<

=====
=====
MATHEMATICAL OBSERVATIONS
=====
=====
* Rank & Degrees of Freedom: For  $m = 5$ ,  $n = 7$ ,  $\text{rank}(A) = 5$ . By Rank-
Nullity Theorem,
 $\dim(\ker(A)) = 7 - 5 = 2$  free variable(s), generating an affine
hyperplane of dimension 2.
* Particular Solution Verification: Setting free variables  $[x_5, x_6]$ 
 $= 0$  produces  $x_p$ 
directly matching the transformed RHS vector in the RREF augmented
column.
* Null Space Basis ( $Ax = 0$ ): The vectors  $v_1$  and  $v_2$  form a linearly
independent basis for  $\ker(A)$ .
For any scalars  $\lambda_1 = 3.14159265$  and  $\lambda_2 = -2.71828182$ ,
 $A(\lambda_1 v_1 + \lambda_2 v_2) = 0$ .
* Precision & Exactness: The maximum discrepancy between  $A*x_{\text{gen}}$  and  $b$ 
is  $\sim 1.24e-14$ ,
which proves numerical stability and correctness of the Gaussian
elimination algorithm.
=====
=====

```